Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосувань»
Практична робота №15
Безпека клієнтських застосунків: типові вразливості та захист

Виконав:
Студент II курсу
групи ІП-56
Гончар І.В.

Перевірив:
Ковтунець О.В.

Київ 2026

1. Мета роботи:

Провести аудит власного застосунку (весь проєкт практикумів 1-14) на три типові клієнтські проблеми - XSS через innerHTML, зайві чи чутливі дані в незахищеному localStorage/IndexedDB, відсутність усвідомлення ролі HTTPS - і усунути щонайменше одну реальну вразливість, продемонструвавши різницю «до» і «після».

2. Хід виконання роботи:

Кроки 1–2. Виявлення та підтвердження XSS-вразливості: Під час аудиту коду застосунку «Каталог настільних ігор» (Варіант 4) було проаналізовано місця виведення даних користувача. Було виявлено, що поле title (назва гри) є потенційним вектором XSS-атаки при використанні небезпечного рендеру, наприклад, директиви v-html у Vue.js (Рис. 1), яка працює аналогічно до innerHTML і не екранує вхідні дані.

```
29
30  // ========================================
31  // VUE КОМПОНЕНТИ ТА ДОДАТОК
32  // ========================================
33  const GameCard = {
34      props: ['id', 'title', 'minPlayers', 'maxPlayers', 'genre', 'img', 'owned', 'isGameOfTheDay'],
35      emits: ['filter', 'toggle-owned'],
36      template: `
37          <article class="card" :class="{ 'card-owned': owned, 'game-of-the-day': isGameOfTheDay }">
38              <h3 v-html="title"></h3>
39              <span class="badge">👥 {{ minPlayers }}-{{ maxPlayers }} гравців</span>
40              <img :src="img" :alt="'Обкладинка гри ' + title">
41              <p class="genre-text" @click="\$emit('filter', genre)">
42                  Жанр: <span class="genre-link">{{ genre }}</span>
43              </p>
44
45              <a :href="'#/games/' + id" class="btn-detail">📖 Детальніше</a>
46
47              <button
48                  :class="['btn-status', owned ? 'btn-played' : 'btn-not-played']"
49                  @click="\$emit('toggle-owned', id)">
50                  {{ owned ? '✅ Зіграно' : '❌ Не зіграно' }}
51              </button>
```

Рис. 1 — Скріншот вразливого коду у файлі script.js

Для підтвердження вразливості у базу даних було додано запис, де назва містила тестовий рядок <img src=x onerror="alert('XSS')">. Після завантаження сторінки браузер відрендерив рядок як HTML, що призвело до успішного виконання шкідливого скрипта та появи спливаючого вікна alert (Рис. 2).

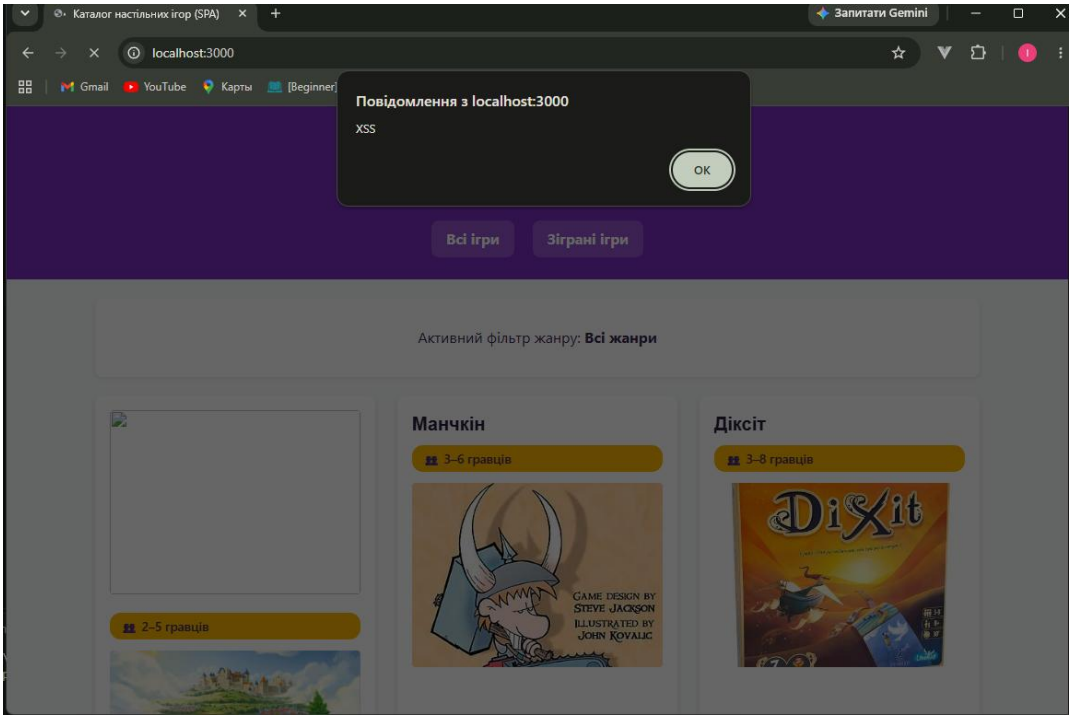


Рис. 2 — Скріншот з alert

Кроки 3–5. Усунення вразливості та перевірка: Щоб закрити XSS-вразливість, виведення поля title було переписано з використанням безпечного синтаксису — вбудованої інтерполяції Vue {{ }} (Рис. 3). Вона є аналогом textContent і завжди трактує рядок виключно як текст.

```
// ========================================
// VUE КОМПОНЕНТИ ТА ДОДАТОК
// ========================================
const GameCard = {
    props: ['id', 'title', 'minPlayers', 'maxPlayers', 'genre', 'img', 'owned', 'isGameOfTheDay'],
    emits: ['filter', 'toggle-owned'],
    template: `
        <article class="card" :class="{ 'card-owned': owned, 'game-of-the-day': isGameOfTheDay }">
            <h3>{{ title }}</h3>
            <span class="badge">👥 {{ minPlayers }}{{ maxPlayers }} гравців</span>
            <img :src="img" :alt="'Обкладинка гри ' + title">
            <p class="genre-text" @click="\$emit('filter', genre)">
                Жанр: <span class="genre-link">{{ genre }}</span>
            </p>

            <a :href="'#/games/' + id" class="btn-detail">📖 Детальніше</a>

            <button
                :class="['btn-status', owned ? 'btn-played' : 'btn-not-played']"
                @click="\$emit('toggle-owned', id)">
                {{ owned ? '✅ Зіграно' : '❌ Не зіграно' }}
            </button>
```

Рис. 3 — Скріншот виправленого коду

Після виправлення той самий тестовий рядок більше не виконується як скрипт, а безпечно відображається на екрані як звичайний текст (Рис. 4). Усі інші місця потенційного виведення даних у проєкті також переведено на безпечний рендер.

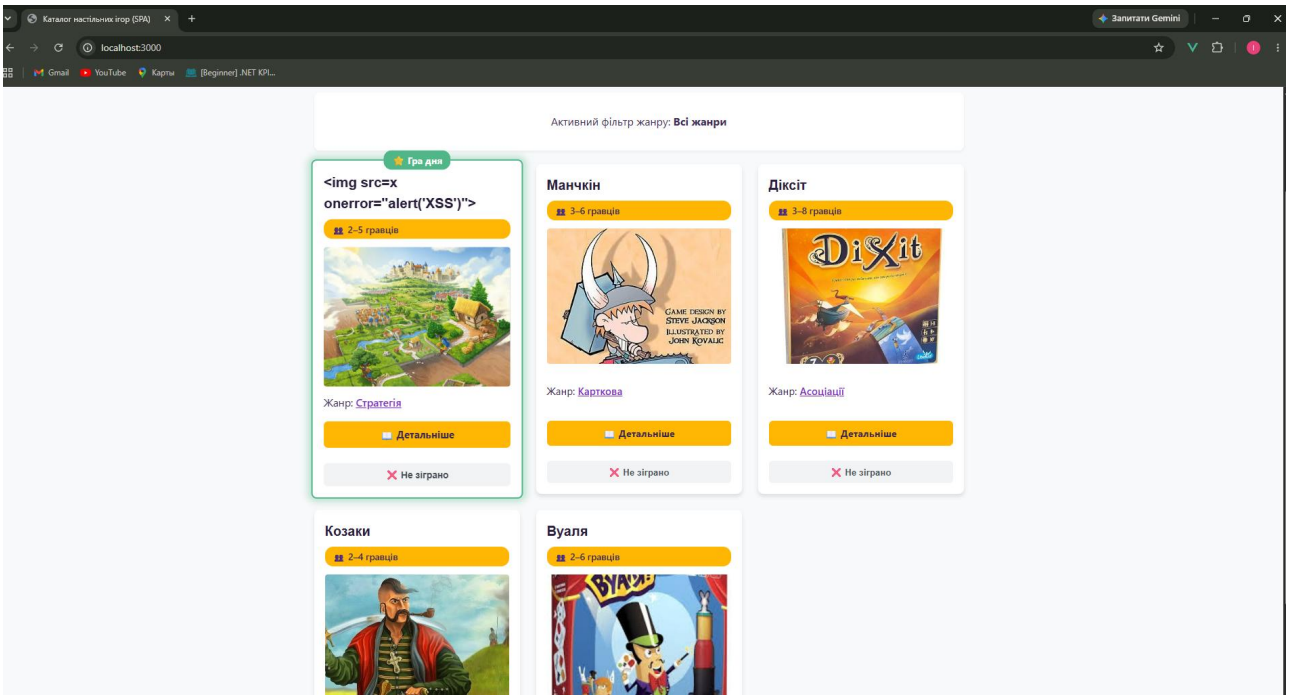


Рис. 4 — Скріншот з безпечним текстом на екрані

Крок 6. Аудит даних у localStorage/IndexedDB: Було перевірено вміст локальних сховищ через вкладку Application у DevTools. Для предметної області «Каталог настільних ігор» сховище (якщо використовується) містить лише базові неперсональні дані: назви ігор, жанри, кількість гравців. Жодної чутливої, особистої інформації чи токенів автентифікації у сховищі немає, тому збереження цих даних у незашифрованому вигляді в межах навчального проєкту є припустимим і безпечним.

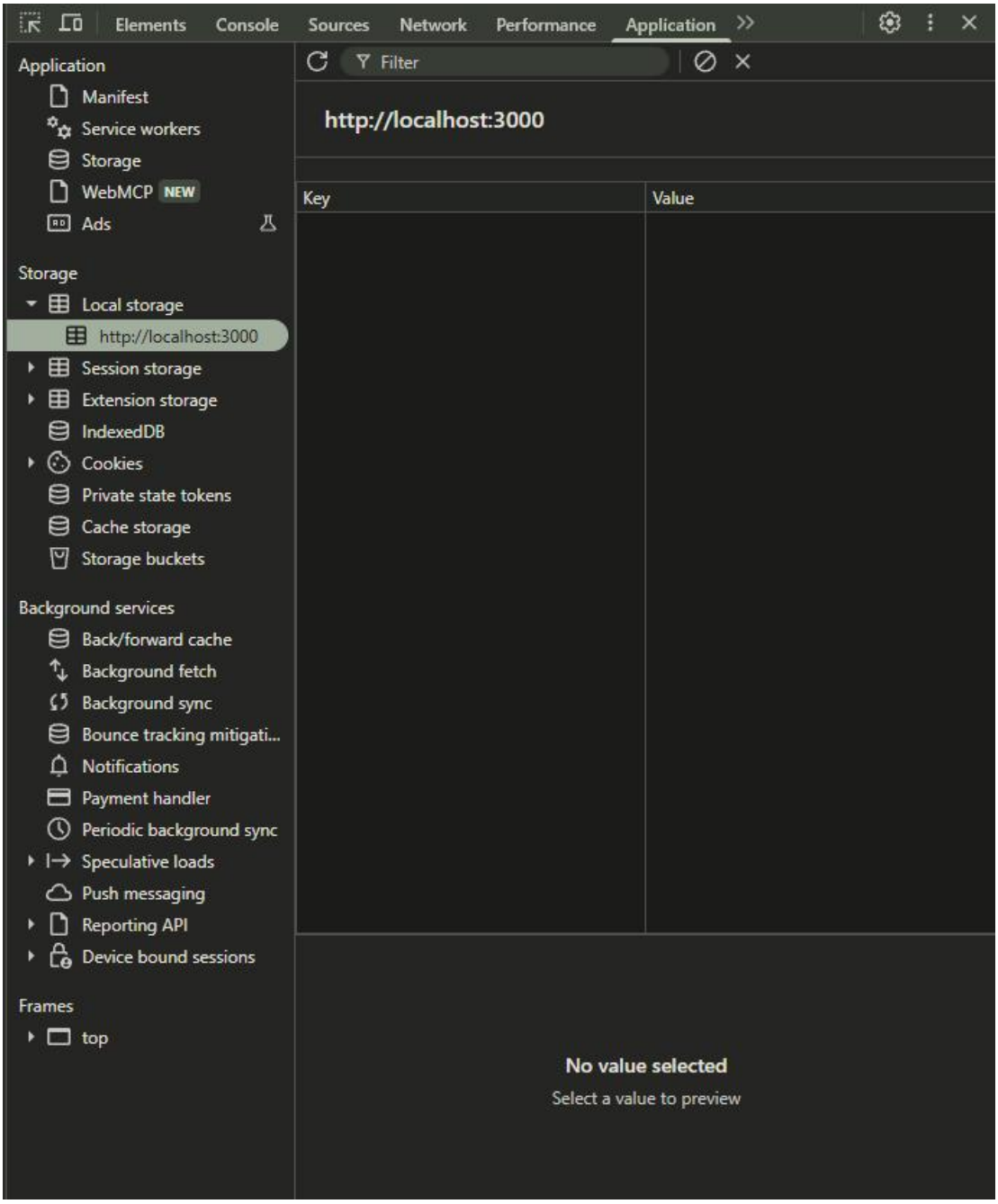


Рис. 5 — Перевірка вмісту Local Storage у DevTools

Кроки 7–8. Перевірка секретів та протокол HTTPS: Клієнтський код перевірено на відсутність захардкоджених секретних ключів чи реальних паролів. Для розгортання проєкту передбачено використання платформи (наприклад, GitHub Pages), яка забезпечує з'єднання через захищений протокол HTTPS за замовчуванням. Це унеможливлює перехоплення чи підміну даних третьою стороною (man-in-the-middle).

Кроки 9–10. Версіонування: Усі зміни з аудиту та безпеки задокументовано та зафіксовано фінальним комітом у репозиторії на платформі GitHub (Рис. 6).

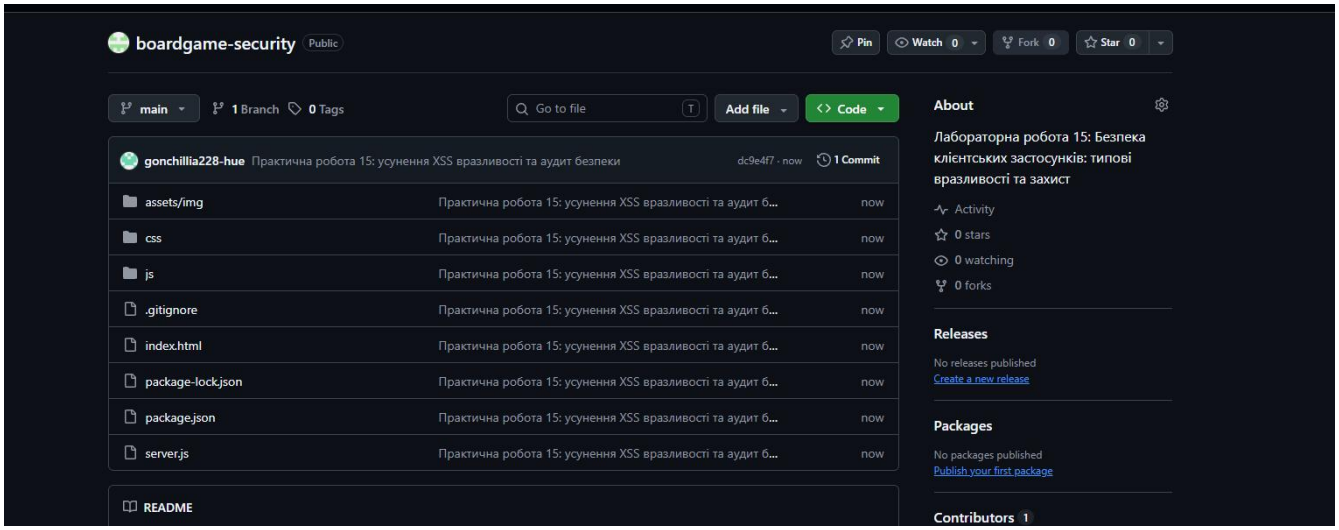


Рис. 6 — Скріншот коміту на GitHub

Посилання на репозиторій:

https://github.com/Bitshkiper218/boardgame-security

Висновок:

Під час виконання практичної роботи №15 було проведено успішний аудит безпеки клієнтської частини вебзастосунку «Каталог настільних ігор». Було виявлено та усунено XSS-вразливість при рендері поля title шляхом заміни небезпечного виведення на безпечний аналог textContent. Аудит локального сховища IndexedDB/localStorage підтвердив відсутність витоку чутливих персональних даних. Також перевірено відсутність захардкоджених секретів у коді та усвідомлено важливість розгортання застосунків із використанням захищеного протоколу HTTPS для запобігання атакам man-in-the-middle.